

Machine Learning Week 9 Assignment Report

By 21337470

Introduction

This is a report of answers to the questions asked and the methods, outputs, hyperparameter adjustments and added code I put into the given code zip file for the assignment. The report on the answers is roughly 6 pages of the pdf with the rest being the appendix of the hyperparameter codes, output of the codes, added testing and evaluation codes and additional fixes I have made to the gpt.py file. Clicking on the hyperlinked CODE APPENDIX lines should bring the reader to the linked code/output that is being discussed in the given section.

Answers 1

- A. For getting the number of words and the unique words from the datasets given, I used code in datasets.py in order to parse through the text datasets by using `text.split()` in the line by line interpretation of the dataset to get the words and then used a `Set()` in python to get the unique words in the dataset. Here is the output from the code:

```
{'File': 'input_childSpeech_trainingSet.txt', 'Total words': 55076, 'Vocabulary Size': 88}  
{'File': 'input_childSpeech_testSet.txt', 'Total words': 5347, 'Vocabulary Size': 88}  
{'File': 'input_shakespeare.txt', 'Total words': 202651, 'Vocabulary Size': 25670}
```

Input_childSpeech_trainingSet: This dataset appears to contain basic sentences with a lot of times, unrelated and improper sentence structure, as the name suggest, it likely mimics speech of children in that sense. The total size/ number of words in the dataset is 55076, with a very limited vocabulary size of 88 unique words in total (again, due to children's limited vocabulary I believe).

Input_childSpeech_testSet: This dataset only has 5347 words with same number of unique words/ vocabulary as the training set, at 88 unique words. The reduced number of overall words and same number of vocabulary is due to this dataset being used as the test dataset for the model that will be trained on the much larger training set dataset.

Input_shakespeare : This dataset seems to be containing a script of a Shakespeare play, this dataset is quite large with about 202651 words and as Shakespeare is known for his large vocabulary, it unsurprisingly contains a large number of unique words/vocabulary 25670.

- B. For downsizing the model, I chose to reduce most of the parameters outlined to get the parameters size down to less than 1 million in size, I have given the relevant code for the parameters in [CODE APPENDIX - 1](#).

My rationale behind reducing the parameters are as follows:

I reduced the **n_embd/Embedding Dimension** from 384 to 128. As the childSpeech training set is being used in this instance, we can safely reduce the context embedding length for the model as the childSpeech training set does not contain proper contextual sentences to properly leverage the embedding dimensions of the model, hence I chose to reduce it and let the model capture less information per token.

I reduced **n_head/Number of Attention Heads** from 6 to 4. Attention heads allow the model to focus on different parts of the input sequence in parallel. More heads mean more parallel computations, but they were also increasing the model size, hence I reduced them.

I also reduced **n_layer/ Number of Transformation Layers**, since the dataset has simple patterns and is not very complex, I felt that reducing the layers would lower the model complexity in accordance to the datasets complexity.

I also reduced the **block size** from 256 to 128. Block size determines the amount of words to process at a time, since the dataset contains simple short sentences, the reduction of this parameter would not affect model accuracy and reduce complexity. (I plan to reduce it more later on since 128 is still high for this simple dataset)

Finally I reduced the **dropout rate** from 0.2 to 0.1 as the dataset is quite small and wont easily run into problems with overfitting.

The output I got was given in [CODE APPENDIX 4](#)

Seeing the output, the train loss and validation loss are very close together, we can interpret the model is not overfitting from this data (if it was then it would do poorly on validation data). We can also observe that the model is able to make a few coherent sentences (like the childSpeech in the dataset) although It does have some repetitions of phrases.

C. Model 2

Parameter [CODE APPENDIX: 2](#)

Now lets look at the changed parameters of this iteration over the previous one:

Batch_size: I changed batch size from 64 to 32 to make the model computations run faster.

Block_size: I changed block size from 128 to 64 as the dataset contains of quite small sentences, hence a smaller block size would fit the data better.

Max_iters: I reduced the max iterations from 5000 to 3000 as in the previous models, the model was converging around iteration 3000 with very few changes in the loss function afterwards.

Eval_interval: I increased it from 100 to 500 so the model trains faster and evaluates less frequently.

Eval_iters: Increased it from 100 to 200 so the model makes better and more accurate loss estimations.

N_head: Increased from 4 to 5 to improve the ability of the model to capture data complexities.

N_layer: Increased from 4 to 5 to allow the model to learn more complex patterns from the data.

Output is given in [CODE APPENDIX: 5](#)

Now looking at this latest output, we can see that this model performs even better than the previous model, with only a difference of 0.02 between the training and testing sets, although the loss is higher than the previous model, this performs better in terms of generalization and preventing overfitting. Looking at the generated text, we can see that there is more diversity in the output and better cohesion, however as before, there seems to be repetitive phrases again.

Model 3

Hyperparameter code, [CODE APPENDIX: 3](#)

Here are the explanation for parameters I changed for this model:

Batch_size: I increased the batch size from 32 to 64 to allow the model to process more sequences in parallel, in order to ensure better gradient updates.

Block_size: I increased the block size from 64 to 128 as the larger context length allows the model to find relationships better in the model, despite the sentences being short, this seems to help in producing more coherent output by taking several sentences in a larger context.

Max_iters: I increased the max iterations from 3000 to 4000 to give the model more time to converge and improve further.

N_embd: I reduced the embedding dimension from 192 to 128 to bring the total model parameters under the 1M threshold.

N_head: I reduced the number of attention heads from 5 to 4 to align with the reduced embedding size (to keep under 1M parameters) and as seen from previous models, that 4 works quite well on this dataset.

Dropout: I increased dropout from 0.15 to 0.2 to reduce the risk of overfitting even further to perhaps reduce repetition as well and improve regularization.

Output given in [CODE APPENDIX: 6](#)

Looking at this latest output, we can see that this model performs similarly well, with a very small difference of 0.0038 between the training and validation losses, showing very good generalization and minimal overfitting. While the loss is slightly higher than the previous model, the improvement in generalization makes this model more reliable. Examining the generated text, there is noticeable diversity in the phrases and better alignment with the child speech patterns and more coherent sentences with better context. However, as with the previous model, repetitive patterns such as "I jump I hide" are still present.

- D. The individual bias terms in the self-attention layers of a transformer model are essentially a series of small values that are added to the calculations. Which small changes/tweaks that

the model applies during training to detect patterns in the data. These tweaks enable the model to adjust more closely to irregularities in the dataset. This flexibility usually helps the model learn the complex relationships in the dataset and without these bias terms, the model may not fully capture these relationships, resulting in worse predictions or reduced diversity in outputs. Including bias terms makes the model more “adaptive” to the data it is trained on, and may even help in generalization of the model as a whole.

The bias terms have been set to false/ been disabled in our training of the models so far, so let's enable them and see the output.

```
self.key = nn.Linear(n_embd, head_size, bias=True)
self.query = nn.Linear(n_embd, head_size, bias=True)
self.value = nn.Linear(n_embd, head_size, bias=True)
```

Output after enabling bias terms (3rd Model Hyperparameters) in [CODE APPENDIX: 7](#)

Comparing these results to the older model 3, The values for loss are comparatively lower, likely due to the bias terms helping the model adapt better to the given set dataset, however these improvements are only marginally better than the model with bias disabled. We can also see in the output that the phrases outputted by the model are more diverse and have more variation in their structure as well, with the repetitive phrases still remaining.

- E. Skip connections in a transformer model are a way for the model to carry information forward through the layers without altering it. Basically, the model takes the input to a layer and adds it directly to the output of that layer. This means that the original data is kept while the layer also learns new patterns. Skipping connections helps the model avoid losing important details as it processes the data. Without skip connections, the model might struggle to learn effectively because some of the original information could get forgotten (due to Vanishing Gradient Problem). It also helps reduce the risk of overfitting and ensures the model can generalize better to unseen data.

The model currently uses skip connections, now by disabling them by modifying the code like this:

```
def forward(self, x):
    x = self.sa(self.ln1(x))
    x = self.ffwd(self.ln2(x))
    return x
```

Now looking at the output, [CODE APPENDIX: 8](#)

The output we get for disabling skip connections, indicates that skip connections is absolutely necessary for the model. The training and test loss are ten times higher than with skip connections, with the losses being super high (about 3.12) and looking at the generated text, we can also see that the model failed to learn even basic words, much less sentences. The generated text is just a random mix of characters which fail to form even words. This is likely because due to the vanishing gradient issue, the model fails to “remember” and to learn any words or sentences from the model.

- A. For evaluating the test loss on the dataset, I added code for evaluating the test loss and the baseline model to compare it against, here is a short summary of what the code does. Testing Code given in [CODE APPENDIX: 9](#)

In the test loss/evaluate_test_loss function, the model is set to evaluation mode using model.eval() to prevent any updates during the process. The test dataset is processed in small batches, where each batch has input tokens and target tokens that are shifted by one position. The model predicts the next tokens for the input batch, and the loss is calculated by comparing these predictions with the actual target tokens. The function then averages the loss across all batches to give the final test loss.

To create the baseline model, it basically chooses the most frequent token from the training dataset by creating fixed predictions, where the most frequent token is given the highest probability(10 in our code) . For each batch of the test dataset, the loss is calculated by comparing these fixed predictions with the actual target tokens, and the average loss across all batches is returned as the baseline test loss.

Now lets look at the output:

[CODE APPENDIX: 10](#)

The output we get for the baseline model confirms that the trained model significantly outperforms the baseline model. The baseline test loss is much higher at 8.1991 compared to our trained model's test loss of 0.3524. This clearly indicates that the baseline's "naïve" approach of always predicting the most frequent token fails to capture the patterns in the data, moreso since it is a speech data which has quite complex relationships. Looking at the trained model's generated text, we see that it has learned to generate coherent sentences, matching closely with our dataset type speech data, while the baseline model would simply repeat the most frequent token(character), producing completely meaningless outputs. Therefore I think we can safely infer that the trained model, which can generate coherent sentences and captures complex relationships in the data, is significantly better than the baseline model which only predicts most common tokens and cannot output or capture any complex sentences/data.

- B. At first, when running the test set of input_shakespeare, I encountered a KeyError as the Shakespeare test set had characters that the model had not seen from the childSpeech dataset. To fix this, I decided to read both datasets and make a combined vocabulary for them before encoding to ensure that the key error does not take place again. (fix is in [CODE APPENDIX: 11](#))

Output given in [CODE APPENDIX: 12](#)

The output show that the trained model performs significantly better than the baseline on the Shakespeare test set, with a loss of 7.5787 compared to the baseline loss of 8.4797. This demonstrates the model's ability to generalize to some extent, even on data it was not trained for. Obviously, the test loss on the Shakespeare data is quite high, due to the model being trained on Childish speech patterns and then having to tackle the eloquent vocabulary of Shakespeare, it cannot accurately predict the type of complex speech patterns that are in

the Shakespeare dataset. The baseline performs poorly as it, like before, naively predicts the most frequent token, as it is unable to capture any complex patterns or words of the dataset.

In question (ii)(a), the loss on the child speech test set was around 0.3524, which is very very much lower than the loss on the Shakespeare dataset. This difference tells us the importance of training and testing on data from the same or similar domains. The trained model performs well on the child speech test set because it shares vocabulary, structure, and patterns with the training set. However, it struggles on Shakespeare due to the completely different nature of the data, although the language is the same, we had to make a fix for the vocabulary integration, and the sentence structure is much more complex. Thus our model trained on the child dataset cannot perform well on the Shakespeare dataset, a suitable analogy for this would be that we are trying to teach a child who has just learnt to speak to read Shakespeare, which would obviously result in the child not being able to comprehend such results.

The result showcases the domain-specific nature of the trained model. The low loss we observed from the child speech test set shows that the model is highly effective when the test data is similar to the training data, as it should be. Now the high loss on the Shakespeare dataset demonstrates that since the model is not trained on the domain close to the complex sentences in the Shakespeare model, it fails to perform well on it.

I think that this pipeline can be used to create models for specific types of data such as interpreting or the outputting text close or related to the speech data it is trained on (such as generating Shakespearean type text if it is trained on a training set of Shakespearean texts). Additionally, it can be used to evaluate how well a model generalizes by testing it on different types of domain data, as we did in (ii) questions.

Code Appendix

1.#Model 1 Hyperparameters

```
batch_size = 64 # how many independent sequences will we process in
parallel?
block_size = 128 # what is the maximum context length for predictions?
max_iters = 5000
eval_interval = 100
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 100
n_embd = 128
n_head = 4
n_layer = 4
dropout = 0.1
```

2. # Model 2 hyperparameters

```
batch_size = 32 # how many independent sequences will we process in
parallel?
block_size = 64 # what is the maximum context length for predictions?
max_iters = 3000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 128
n_head = 5
n_layer = 5
```

```
dropout = 0.15
3.# Model 3 hyperparameters
batch_size = 64 # how many independent sequences will we process in
parallel?
block_size = 128 # what is the maximum context length for predictions?
max_iters = 4000
eval_interval = 500
learning_rate = 3e-4
device = 'cuda' if torch.cuda.is_available() else 'cpu'
eval_iters = 200
n_embd = 128
n_head = 4
n_layer = 5
dropout = 0.2
```

4. #Output of model 1

step 4999: train loss 0.3275, val loss 0.3529

Generating training batch

Loss at iteration 4999: 0.3329

Iteration 4999 completed.

Shoes on Look moon

More more No like it

Look moon Bath time

No stop Shoes on

I make mess Uh oh spill

Go park More bubbles

No touch Daddy read me dinosaur book before bed

More more Daddy play

My teddy No stop

Where balll Night night

I sing What is that

Look moon I want cookie
Where kitty go I draw
Big truck My teddy
Where ball No nap
No mine Where ball
I see doggy Yummy apple
No touch Big truck
Mama I want play with big red ball outside Mama I want play with big
red ball outside
More bubbles No s

5. #Output of model 2

step 2999: train loss 0.3725, val loss 0.3745

Generating training batch

Loss at iteration 2999: 0.3583

Iteration 2999 completed.

Where ball I did it
No nap Big truck
Big hug I jump
No like it I dance
I jump Where kitty go
I hungry I dralaw
More more I found it
I run fast Go park
More bubbles More more
Go park I dance
Daddy play No stop
I love you I want more juice please
Where kitty go I want that
I run fast I draw
Big hug No nap

All done I hide
More bubbles I draw
What is that I hide
Night night No touch
Where ball I run fast
I draw Help me please
More bubbbles My teddy
Uh oh spill I hide
Come here I want more juice please

6. #Output of model 3

step 3999: train loss 0.3420, val loss 0.3458

Generating training batch

Loss at iteration 3999: 0.3466

Iteration 3999 completed.

I jump Saw big fluffy doggy at park it run fast
Where ball Big truck
I want cookie Big truck
I want that I tired
Come here More more
Bath time I hungry
My teddy No stop
Big hug No touch
I jump I hide
I hungry All done
I wash hands I want more juice please
Uh oh spill I hide
Help me please I make mess
All done Read book
What is that Daddy read me dinosaur book before bed
I did it I want cookie

Where ball I wash hands
I want that I dance
I found it Shoes on
I jump I sing
I love you More bubbles
Mo

7. #Output of model 3 with bias enabled

step 3999: train loss 0.3414, val loss 0.3454
Generating training batch
Loss at iteration 3999: 0.3537
Iteration 3999 completed.

Saw big fluffy doggy at park it run fast
I hide Go park
Big hug Go park
I jump All gone
Go park Read book
All gone I youngre I hungry
I dance Mama I want play with big red ball outside
Bath time Mama I want play with big red ball outside
I love you Read book
Big truck I want more juice please
Read book All done
Shoes on No mine
Go park I make mess
I see bird I hungry
I jump high Where kitty go
Bath time I love you
I want that I draw
Bath time I dance

I want more juice please Mama I want play wit

8. # Output of Model 3 without skip connections

step 3999: train loss 3.1252, val loss 3.1293

Generating training batch

Loss at iteration 3999: 3.1292

Iteration 3999 completed.

ltns tmocM as arhesaod CNubi ekt soe l
cghul
ebnio
lpt ckecNdeomph m lsnafa ea k ll ynRSnnr e
ipipoealo
huoyI e ge iNio hHfakin cslelrLkirogrpcelIaeioekiieefpAen f i
f
unnkefejtahtBnnsrb
poeyh Ik
gAanupiw
djf niu ewji gyat r tl
ksv ghMrmIpirad ejeshuIa dwooe ai rrtmboNbl eu o RddipyWsbtriplpi i
temnbsa ahp
cIa gwetst wu
dmoagadbbctl kalmg a lephthelimi p mk khatgeaggaa k oe feubr w
otfboiendmsp Ia ereoBu Icy mbb
kb eu moIy
rrIs

h
mbptgeeo wophea epdl a drg idodeemki

9. # Code for test sets

```

@torch.no_grad()
def evaluate_test_loss(model, test_data):
    model.eval()
    total_loss = 0
    batch_count = 0
    for i in range(0, len(test_data) - block_size, block_size):
        x = test_data[i:i+block_size].unsqueeze(0).to(device)
        y = test_data[i+1:i+block_size+1].unsqueeze(0).to(device)
        _, loss = model(x, y)
        total_loss += loss.item()
        batch_count += 1
    return total_loss / batch_count if batch_count > 0 else
float('inf')

# Loading test dataset
with open('input_childSpeech_testSet.txt', 'r', encoding='utf-8') as
f:
    test_text = f.read()
test_data = torch.tensor(encode(test_text), dtype=torch.long)

# Calculating test loss
test_loss = evaluate_test_loss(model, test_data)
print(f"Test Loss: {test_loss:.4f}")

# Baseline model
@torch.no_grad()
def baseline_loss(test_data, most_frequent_token, vocab_size):
    total_loss = 0
    batch_count = 0
    dummy_logits = torch.full((block_size, vocab_size), float('-inf'),
device=device)
    dummy_logits[:, most_frequent_token] = 0

```

```

    for i in range(0, len(test_data) - block_size, block_size):
        y = test_data[i+1:i+block_size+1].to(device)
        loss = F.cross_entropy(dummy_logits, y, reduction='mean')
        total_loss += loss.item()
        batch_count += 1

    return total_loss / batch_count if batch_count > 0 else
float('inf')

```

```

most_frequent_token = torch.mode(train_data).values.item()

```

```

# Baseline model test loss

```

```

baseline_test_loss = baseline_loss(test_data, most_frequent_token,
vocab_size=len(set(data.tolist()))))
print(f"Baseline Test Loss: {baseline_test_loss:.4f}")

```

10. step 3999: train loss 0.3414, val loss 0.3454

Generating training batch

Loss at iteration 3999: 0.3537

Iteration 3999 completed.

Saw big fluffy doggy at park it run fast

I hide Go park

Big hug Go park

I jump All gone

Go park Read book

All gone I youngre I hungry

I dance Mama I want play with big red ball outside

Bath time Mama I want play with big red ball outside

I love you Read book

Big truck I want more juice please

Read book All done

Shoes on No mine

Go park I make mess
I see bird I hungry
I jump high Where kitty go
Bath time I love you
I want that I draw
Bath time I dance
I want more juice please Mama I want play wit

Test Loss: 0.3524

Baseline Test Loss: 8.1991

11.# Added code for fixing KeyError

```
with open('input_childSpeech_trainingSet.txt', 'r', encoding='utf-8')  
as f1, open('input_shakespeare.txt', 'r', encoding='utf-8') as f2:
```

```
    text = f1.read()
```

```
    shakespeare_text = f2.read()
```

Combine both texts

```
combined_text = text + shakespeare_text
```

```
chars = sorted(list(set(combined_text)))
```

```
vocab_size = len(chars)
```

Updating mappings

```
stoi = {ch: i for i, ch in enumerate(chars)}
```

```
itos = {i: ch for i, ch in enumerate(chars)}
```

```
encode = lambda s: [stoi[c] for c in s]
```

```
decode = lambda l: ''.join([itos[i] for i in l])
```

```
test_data = torch.tensor(encode(shakespeare_text), dtype=torch.long)
```

12.# Output of Model 3 on test loss of Shakespeare dataset

step 3999: train loss 0.3407, val loss 0.3457

Generating training batch

Loss at iteration 3999: 0.3591

Iteration 3999 completed.

I climb What is that

I want that Saw big fluffy doggy at park it run fast

I hide Uh oh spill

Shoes on Where kitty go

My teddy I dance

I want more juice please I climb

I dance Big hug

I hide I see bird

I love you All done

I hide No touch

Look moon No mine

I dance Help me please

Big truck Come here

Yummy apple Big truck I tired

Look airplane What is that No stop

I make mess No like it

I sing Big truck

Daddy read me dinosaur book before bed I climb

More bubbles Help me please

Go park Bath time

All

Shakespeare Test SetLoss: 7.5787

Baseline Test Loss : 8.4797